

Deep Q-learning

The Q-learning method that we have just covered solves the issue of iteration over the full set of states, but still can struggle with situations when the count of the observable set of states is very large. For example, Atari games can have a large variety of different screens, so if we decide to use raw pixels as individual states, we will quickly realize that we have too many states to track and approximate values for.

In some environments, the count of different observable states could be almost infinite. For example, in CartPole, the environment gives us a state that is four floating point numbers. The number of value combinations is finite (they're represented as bits), but this number is extremely large. We could create some bins to discretize those values, but this often creates more problems than it solves: we would need to decide what ranges of parameters are important to distinguish as different states and what ranges could be clustered together.

In the case of Atari, one single pixel change doesn't make much difference, so it's efficient to treat both images as a single state. However, we still need to distinguish some of the states.

The following image shows two different situations in a game of Pong. We're playing against the artificial intelligence (AI) opponent by controlling a paddle (our paddle is on the right, whereas our opponent's is on the left). The objective of the game is to get the bouncing ball past our opponent's paddle, while preventing the ball from getting past our paddle. We can consider the two situations to be completely different: in the right-hand situation, the ball is close to the opponent, so we can relax and watch. However, the situation on the left is more demanding: assuming that the ball is moving from left to right, the ball is moving toward our side, so we need to move our paddle quickly to avoid losing a point. The situations in *Figure 6.2* are just two from the 10^{70802} possible situations, but we want our agent to act on them differently.

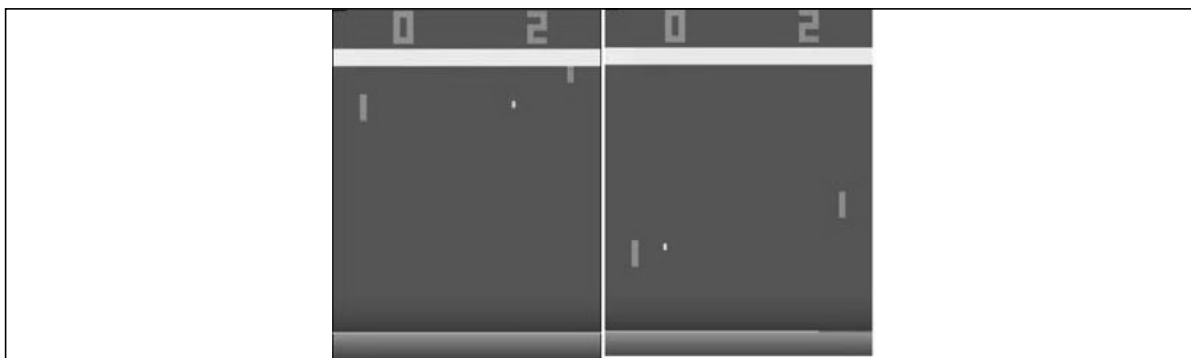


Figure 6.2: The ambiguity of observations in Pong. In the left image, the ball is moving to the right toward our paddle, and on the right, its direction is opposite

As a solution to this problem, we can use a nonlinear representation that maps both the state and action onto a value. In machine learning, this is called a "regression problem." The concrete way to represent and train such a representation can vary, but as you may have already guessed from this section's title, using a deep NN is one of the most popular options, especially when dealing with observations represented as screen images. With this in mind, let's make modifications to the Q-learning algorithm:

1. Initialize $Q(s, a)$ with some initial approximation.
2. By interacting with the environment, obtain the tuple (s, a, r, s') .
3. Calculate loss: $\mathcal{L} = (Q(s, a) - r)^2$ if the episode has ended, or

$$\mathcal{L} = \left(Q(s, a) - \left(r + \gamma \max_{a' \in A} Q(s', a') \right) \right)^2$$
 otherwise.
4. Update $Q(s, a)$ using the **stochastic gradient descent (SGD)** algorithm, by minimizing the loss with respect to the model parameters.
5. Repeat from step 2 until converged.

The preceding algorithm looks simple, but, unfortunately, it won't work very well. Let's discuss what could go wrong.

Interaction with the environment

First of all, we need to interact with the environment somehow to receive data to train on. In simple environments, such as FrozenLake, we can act randomly, but is this the best strategy to use? Imagine the game of Pong. What's the probability of winning a single point by randomly moving the paddle? It's not zero, but it's extremely small, which just means that we will need to wait for a very long time for such a rare situation. As an alternative, we can use our Q-function approximation as a source of behavior (as we did before in the value iteration method, when we remembered our experience during testing).

If our representation of Q is good, then the experience that we get from the environment will show the agent relevant data to train on. However, we're in trouble when our approximation is not perfect (at the beginning of the training, for example). In such a case, our agent can be stuck with bad actions for some states without ever trying to behave differently. This is the *exploration versus exploitation* dilemma mentioned briefly in *Chapter 1, What Is Reinforcement Learning?*, and earlier in this chapter. On the one hand, our agent needs to explore the environment to build a complete picture of transitions and action outcomes. On the other hand, we should use interaction with the environment efficiently: we shouldn't waste time by randomly trying actions that we have already tried and have learned outcomes for.

As you can see, random behavior is better at the beginning of the training when our Q approximation is bad, as it gives us more uniformly distributed information about the environment states. As our training progresses, random behavior becomes inefficient, and we want to fall back to our Q approximation to decide how to act.

A method that performs such a mix of two extreme behaviors is known as an **epsilon-greedy method**, which just means switching between random and Q policy using the probability hyperparameter ϵ . By varying ϵ , we can select the ratio of random actions. The usual practice is to start with $\epsilon = 1.0$ (100% random actions) and slowly decrease it to some small value, such as 5% or 2% random actions. Using an epsilon-greedy method helps us both to explore the environment in the beginning and stick to good policy at the end of the training. There are other solutions to the exploration versus exploitation problem, and we will discuss some of them in the third part of the book. This problem is one of the fundamental open questions in RL and an active area of research that is not even close to being resolved completely.

SGD optimization

The core of our Q -learning procedure is borrowed from supervised learning. Indeed, we are trying to approximate a complex, nonlinear function, $Q(s, a)$, with an NN. To do this, we must calculate targets for this function using the Bellman equation and then pretend that we have a supervised learning problem at hand. That's okay, but one of the fundamental requirements for SGD optimization is that the training data is **independent and identically distributed** (frequently abbreviated as **i.i.d.**).

In our case, data that we are going to use for the SGD update doesn't fulfill these criteria:

1. Our samples are not independent. Even if we accumulate a large batch of data samples, they will all be very close to each other, as they will belong to the same episode.
2. Distribution of our training data won't be identical to samples provided by the optimal policy that we want to learn. Data that we have will be a result of some other policy (our current policy, random, or both in the case of epsilon-greedy), but we don't want to learn how to play randomly: we want an optimal policy with the best reward.

To deal with this nuisance, we usually need to use a large buffer of our past experience and sample training data from it, instead of using our latest experience. This technique is called **replay buffer**. The simplest implementation is a buffer of fixed size, with new data added to the end of the buffer so that it pushes the oldest experience out of it.

Replay buffer allows us to train on more-or-less independent data, but the data will still be fresh enough to train on samples generated by our recent policy. In the next chapter, we will check another kind of replay buffer: prioritized, which provides a more sophisticated sampling approach.

Correlation between steps

Another practical issue with the default training procedure is also related to the lack of i.i.d data, but in a slightly different manner. The Bellman equation provides us with the value of $Q(s, a)$ via $Q(s', a')$ (this process is called *bootstrapping*). However, both the states s and s' have only one step between them. This makes them very similar, and it's very hard for NNs to distinguish between them. When we perform an update of our NN's parameters to make $Q(s, a)$ closer to the desired result, we can indirectly alter the value produced for $Q(s', a')$ and other states nearby. This can make our training very unstable, like chasing our own tail: when we update Q for state s , then on subsequent states we will discover that $Q(s', a')$ becomes worse, but attempts to update it can spoil our $Q(s, a)$ approximation, and so on.

To make training more stable, there is a trick, called *target network*, by which we keep a copy of our network and use it for the $Q(s', a')$ value in the Bellman equation. This network is synchronized with our main network only periodically, for example, once in N steps (where N is usually quite a large hyperparameter, such as 1k or 10k training iterations).

The Markov property

Our RL methods use Markov decision process (MDP) formalism as their basis, which assumes that the environment obeys the Markov property: observations from the environment are all that we need to act optimally. (In other words, our observations allow us to distinguish states from one another.)

As you saw from the preceding Pong screenshot, one single image from the Atari game is not enough to capture all the important information (using only one image, we have no idea about the speed and direction of objects, like the ball and our opponent's paddle). This obviously violates the Markov property and moves our single-frame Pong environment into the area of **partially observable MDPs (POMDPs)**. A POMDP is basically MDP without the Markov property, and it is very important in practice. For example, for most card games in which you don't see your opponents' cards, game observations are POMDPs, because the current observation (your cards and cards on the table) could correspond to different cards in your opponents' hands.

We won't discuss POMDPs in detail in this book, but we will use a small technique to push our environment back into the MDP domain. The solution is maintaining several observations from the past and using them as a state. In the case of Atari games, we usually stack k subsequent frames together and use them as the observation at every state. This allows our agent to deduct the dynamics of the current state, for instance, to get the speed of the ball and its direction. The usual "classical" number of k for Atari is four. Of course, it's just a hack, as there can be longer dependencies in the environment, but for most of the games, it works well.

The final form of DQN training

There are many more tips and tricks that researchers have discovered to make DQN training more stable and efficient, and we will cover the best of them in the next chapter. However, epsilon-greedy, replay buffer, and target network form the basis that has allowed the AI company DeepMind to successfully train a DQN on a set of 49 Atari games and demonstrate the efficiency of this approach when applied to complicated environments.

The original paper (without target network) was published at the end of 2013 (*Playing Atari with Deep Reinforcement Learning*, 1312.5602v1, Mnih and others) and used seven games for testing. Later, at the beginning of 2015, a revised version of the article, with 49 different games, was published in *Nature* (*Human-Level Control Through Deep Reinforcement Learning*, doi:10.1038/nature14236, Mnih and others).

The algorithm for DQN from the preceding papers has the following steps:

1. Initialize the parameters for $Q(s, a)$ and $\hat{Q}(s, a)$ with random weights, $\epsilon \leftarrow 1.0$, and empty the replay buffer.
2. With probability ϵ , select a random action, a ; otherwise, $a = \arg \max_a Q(s, a)$.
3. Execute action a in an emulator and observe the reward, r , and the next state, s' .
4. Store transition (s, a, r, s') in the replay buffer.
5. Sample a random mini-batch of transitions from the replay buffer.
6. For every transition in the buffer, calculate target $y = r$ if the episode has ended at this step, or $y = r + \gamma \max_{a' \in A} \hat{Q}(s', a')$ otherwise.
7. Calculate loss: $\mathcal{L} = (Q(s, a) - y)^2$.
8. Update $Q(s, a)$ using the SGD algorithm by minimizing the loss in respect to the model parameters.
9. Every N steps, copy weights from Q to $\hat{Q}$.
10. Repeat from step 2 until converged.

Let's implement it now and try to beat some of the Atari games!

DQN on Pong

Before we jump into the code, some introduction is needed. Our examples are becoming increasingly challenging and complex, which is not surprising, as the complexity of the problems that we are trying to tackle is also growing. The examples are as simple and concise as possible, but some of the code may be difficult to understand at first.

Another thing to note is performance. Our previous examples for FrozenLake, or CartPole, were not demanding from a performance perspective, as observations were small, NN parameters were tiny, and shaving off extra milliseconds in the training loop wasn't important. However, from now on, that's not the case. One single observation from the Atari environment is 100k values, which have to be rescaled, converted to floats, and stored in the replay buffer. One extra copy of this data array can cost you training speed, which will not be seconds and minutes anymore, but could be hours on even the fastest graphics processing unit (GPU) available.

The NN training loop could also be a bottleneck. Of course, RL models are not as huge monsters as state-of-the-art ImageNet models, but even the DQN model from 2015 has more than 1.5M parameters, which is a lot for a GPU to crunch. So, to make a long story short, performance matters, especially when you are experimenting with hyperparameters and need to wait not for a single model to train, but for dozens of them.

PyTorch is quite expressive, so more-or-less efficient processing code could look much less cryptic than optimized TensorFlow graphs, but there is still a significant opportunity for doing things slowly and making mistakes. For example, a naïve version of DQN loss computation, which loops over every batch sample, is about two times slower than a parallel version. However, a single extra copy of the data batch could make the speed of the same code 13 times slower, which is quite significant.

This example has been split into three modules due to its length, logical structure, and reusability. The modules are as follows:

- `Chapter06/lib/wrappers.py`: These are Atari environment wrappers, mostly taken from the OpenAI **Baselines** project.
- `Chapter06/lib/dqn_model.py`: This is the DQN NN layer, with the same architecture as the DeepMind DQN from the *Nature* paper.
- `Chapter06/02_dqn_pong.py`: This is the main module, with the training loop, loss function calculation, and experience replay buffer.

Wrappers

Tackling Atari games with RL is quite demanding from a resource perspective. To make things faster, several transformations are applied to the Atari platform interaction, which are described in DeepMind's paper. Some of these transformations influence only performance, but some address Atari platform features that make learning long and unstable. Transformations are usually implemented as OpenAI Gym wrappers of various kinds. The full list is quite lengthy and there are several implementations of the same wrappers in various sources. My personal favorite is in the OpenAI Baselines repository, which is a set of RL methods and algorithms implemented in TensorFlow and applied to popular benchmarks to establish the common ground for comparing methods. The repository is available from <https://github.com/openai/baselines>, and wrappers are available in this file: https://github.com/openai/baselines/blob/master/baselines/common/atari_wrappers.py.

The list of the most popular Atari transformations used by RL researchers includes:

- *Converting individual lives in the game into separate episodes.* In general, an episode contains all the steps from the beginning of the game until the "Game over" screen appears, which can last for thousands of game steps (observations and actions). Usually, in arcade games, the player is given several lives, which provide several attempts in the game. This transformation splits a full episode into individual small episodes for every life that a player has. Not all games support this feature (for example, Pong doesn't), but for the supported environments, it usually helps to speed up convergence, as our episodes become shorter.
- *At the beginning of the game, performing a random amount (up to 30) of no-op actions.* This skips intro screens in some Atari games, which are not relevant for the gameplay.
- *Making an action decision every K steps, where K is usually 4 or 3.* On intermediate frames, the chosen action is simply repeated. This allows training to speed up significantly, as processing every frame with an NN is quite a demanding operation, but the difference between consequent frames is usually minor.
- *Taking the maximum of every pixel in the last two frames and using it as an observation.* Some Atari games have a flickering effect, which is due to the platform's limitation. (Atari has a limited number of sprites that can be shown on a single frame.) For the human eye, such quick changes are not visible, but they can confuse NNs.